Connecting Django to PostgreSQL

A simple step-by-step guide — VU Star Project (Day 1)

What this guide covers

This guide explains, in simple words, how we connected a Django project to a PostgreSQL database. It covers creating the database, creating a database user, giving that user the right permissions, and telling Django how to connect to it.

Step 1: Install PostgreSQL and pgAdmin

PostgreSQL is the database system that stores our data. pgAdmin is a visual tool (a program with buttons and menus) that lets us manage PostgreSQL without typing commands. If PostgreSQL is installed, pgAdmin usually comes with it.

Step 2: Create a Database

A database is like a container that will hold all our project's data (tables, records, etc). We created one using pgAdmin:

1. Open pgAdmin and expand your PostgreSQL server.
2. Right-click on **Databases** and choose **Create > Database...**
3. Name it **vustars_db** and save.

Step 3: Create a Database User

Instead of using the default admin account for everything, we created a separate user just for this project. This is safer and is the standard professional practice.

1. Right-click **Login/Group Roles** and choose **Create > Login/Group Role...**
2. Under the **General** tab, name it **vustars_user**.
3. Under the **Definition** tab, set a password.
4. Under the **Privileges** tab, turn ON "Can login?" — otherwise the user cannot connect at all.
5. Save.

Step 4: Give the User Access to the Database

Creating a user does not automatically give them permission to use a specific database. We had to grant that access.

1. Right-click **vustars_db** > **Properties** > **Security** tab.

2. Click the "+" button to add a new row.
3. Set **Grantee** to **vustars_user** and **Privileges** to **ALL**.
4. Save.

Step 5: Fix the "Permission Denied" Error (Important)

Newer versions of PostgreSQL (15 and above) add an extra layer of protection: even after giving a user access to a database, that user still cannot create tables inside it by default. This caused an error that said *"permission denied for schema public"* when Django tried to set itself up.

We fixed this by running the following command in pgAdmin's Query Tool (opened while connected to vustars_db):

```
GRANT ALL PRIVILEGES ON SCHEMA public TO vustars_user;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO vustars_user;
ALTER SCHEMA public OWNER TO vustars_user;
```

> In simple words: this tells PostgreSQL, "allow vustars_user to fully create and manage things inside this database."

Step 6: Store the Connection Details Safely (.env file)

We never write passwords or secret keys directly inside our code, because that code is later uploaded to GitHub for the world to see. Instead, we store this sensitive information in a separate file called **.env**, which is never uploaded (it is listed in .gitignore).

Our **.env** file (inside the backend folder, next to manage.py) looks like this:

```
SECRET_KEY=your-django-secret-key
DEBUG=True
DB_NAME=vustars_db
DB_USER=vustars_user
DB_PASSWORD=your-password
DB_HOST=localhost
DB_PORT=5432
```

Step 7: Tell Django to Use These Details (settings.py)

In the project's settings.py file, we used a small library called python-decouple to read the values from the .env file, and gave those values to Django's DATABASES setting:

```
from decouple import config

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': config('DB_NAME'),
        'USER': config('DB_USER'),
```

```
        'PASSWORD': config('DB_PASSWORD'),
        'HOST': config('DB_HOST'),
        'PORT': config('DB_PORT'),
    }
}
```

Step 8: Run Migrations

Finally, we ran this command to let Django create its required tables inside our new PostgreSQL database:

```
python manage.py migrate
```

When this command finishes without errors, it means Django is successfully talking to PostgreSQL, and the database now has real tables inside it.

Quick Summary

- Installed PostgreSQL and used pgAdmin to manage it visually.
- Created a database (vustars_db) and a dedicated user (vustars_user).
- Gave the user permission to access and build inside the database.
- Fixed a common PostgreSQL 15+ permission issue with a GRANT command.
- Kept all secret details in a .env file, kept out of GitHub.
- Connected Django to PostgreSQL through settings.py.
- Ran migrations to confirm everything works.